研究论文

生成式人工智能辅助图形化 Python 编程教学平台的设计与实现

——以 Visual Python 为例

Design and Implementation of a Generative AI-Assisted Visual Programming Platform for Python Teaching: A Case Study of Visual Python

作者单位:	重庆市育才中学校
作　　者:	谭　伟
完成日期:	2026 年 10 月

2026 年 10 月

摘　要

图形化编程降低了编程入门门槛，但以 Scratch 为代表的工具与以 Python 为代表的文本语言之间存在迁移鸿沟；生成式人工智能编程助手面向专业开发者直接生成代码文本，对初学者是难以审视与迁移的“黑箱”；创客与物联网教育中的硬件创作又长期依赖专业开发工具。针对上述矛盾，本文设计并实现了桌面级图形化编程教学平台 Visual Python。

平台以积木为编辑媒介，统一支撑 Python（含 Manim 数学动画）与三类 MicroPython 开发板（ESP32-S3、ESP32-WROOM、ESP8266-ESP12E）共四类工作台，构建“用户界面—前端核心—桌面服务—本地运行时与硬件”四层架构，以 79 项桌面命令贯通本地 Python 执行、Manim 渲染、语音配音、MicroPython 固件烧录、串口会话与离线硬件模拟。平台将大模型智能体内嵌于桌面端，提出两项核心协同机制：其一，采用“代码优先、积木落地”的生成路径——智能体先生成完整的 Python 或 MicroPython 程序，再由确定性转换器将其解析为真实积木树，经隔离工作区校验积木上下文与反向生成代码后一次提交；转换器无法覆盖时回退到结构化积木树。代码在此仅作为可编译的中间表示，而非最终交付物，使 AI 生成既具备文本生成的覆盖率，又保持积木结果可见、可审查、可撤销。其二，智能体可依据学习者需求自主开发插件——经“规划—中间表示编译—静态验证—宿主运行检查—用户确认安装—失败回滚”闭环产出新的积木包，使平台能力可在运行期扩展。

平台实现 601 种积木（含 144 个 Manim 积木、119 个硬件积木与 NumPy、Pandas、Matplotlib 积木）、127 个自动化测试文件（1285 项用例），并构建了覆盖四类工作台的 32 项 AI 积木生成基准任务；在 qwen3.8-flash 上的首轮诊断性实测通过 12 项（37.5%），据此定位了转换器覆盖与长任务稳定性两类主要缺口。ESP32-WROOM 与 ESP8266 真机冒烟测试通过。入门案例实测中，示例场景动画渲染耗时 75 秒。平台可为程序设计、数学可视化与创客教育提供一体化中文教学环境，也为“可解释的人工智能辅助编程学习”提供了可复用的设计范式。

关键词：图形化编程；生成式人工智能；Python 教学；代码到积木转换；智能体；插件体系；创客教育

Abstract

Visual programming lowers the threshold of learning to program, yet a transfer gap remains between block-based tools such as Scratch and text-based languages such as Python; generative AI coding assistants produce code text directly and thus stay opaque to novices; and hardware creation in maker education still relies on professional toolchains. This paper presents Visual Python, a desktop visual programming platform that unifies four workbenches - Python with Manim animation and three MicroPython boards (ESP32-S3, ESP32-WROOM, and ESP8266-ESP12E) - under a four-layer architecture with 79 desktop commands covering local Python execution, Manim rendering, voiceover synthesis, MicroPython firmware flashing, serial sessions, and offline hardware simulation.

An embedded large-model agent collaborates through two mechanisms. First, a code-first, block-grounded generation path: the agent writes a complete Python or MicroPython program, a deterministic converter lowers it into a real Blockly tree, and the host validates block contexts and the regenerated code in an isolated workspace before committing once; when the converter cannot cover a construct, the agent falls back to a structured block tree. Code is treated as an intermediate representation rather than the final artifact, so generation inherits code-level coverage while remaining visible, inspectable, and undoable at the block level. Second, the agent can autonomously author plugins on demand through a closed loop of planning, intermediate-representation compilation, static validation, in-host runtime checks, user-confirmed installation, and rollback on failure. The platform implements 601 block types (including 144 Manim blocks and 119 hardware blocks) with 127 automated test files (1,285 cases), and defines a 32-task benchmark for AI block generation across the four workbenches; an initial diagnostic run with qwen3.8-flash passed 12 of 32 tasks (37.5%), exposing converter-coverage and long-task-stability gaps, while smoke tests passed on real ESP32-WROOM and ESP8266 boards.

In a hands-on trial, the example animation scene rendered in 75 seconds. The platform offers an integrated Chinese teaching environment for programming, mathematical visualization, and maker education, and a reusable design paradigm for explainable AI-assisted programming education.

Key words: visual programming; generative artificial intelligence; Python teaching; code-to-block conversion; agent; plugin architecture; maker education

目　录

摘　要 I

Abstract II

插图和附表清单 V

符号和缩略词说明 VI

第一章　绪论 1

1.1　研究背景与意义 1

1.2　研究现状与问题提出 1

1.3　研究内容与创新点 2

1.4　论文组织结构 3

第二章　相关工作 4

2.1　图形化编程与积木—代码过渡 4

2.2　生成式人工智能辅助编程与创客硬件 4

2.3　研究现状述评 4

第三章　平台设计 6

3.1　设计目标与原则 6

3.2　总体架构 6

3.3　积木体系与代码生成 7

3.4　积木编辑协同机制设计 8

3.5　AI 自主插件开发机制设计 8

3.6　评估指标设计 9

第四章　平台实现与典型案例 10

4.1　实现规模与质量保障 10

4.2　工作台交互实现 10

4.3 AI 辅助积木编写实现 11
4.4 AI 插件开发实现 11
4.5 动画渲染与配音工作流 12
4.6 硬件编程工作流 12
4.7 教学应用模式 13
4.8 插件体系与生态 13
第五章 讨论 15
5.1 特色分析 15
5.2 与典型工具的能力对比 15
5.3 有效性威胁与局限 16
第六章 结论与展望 17
6.1 研究结论 17
6.2 主要贡献 17
6.3 研究局限与未来工作 17
致 谢 18
声 明 18
参考文献 18
附录 A 平台实现度量明细 19

插图和附表清单

插图清单：

图 3-1　Visual Python 平台总体架构

图 4-1　Visual Python 工作台分屏视图：积木与 Python 代码实时对照

图 4-2　AI 辅助积木编写工作流

图 4-3　AI 插件开发流程

图 4-4　人机协同图形化编程教学模式

附表清单：

表 3-1　积木体系构成

表 5-1　本平台与典型编程工具能力对比

表 A-1　平台实现度量明细

符号和缩略词说明

缩略词	英文全称	中文释义
AI	Artificial Intelligence	人工智能
LLM	Large Language Model	大语言模型
C2B	Code-to-Block	代码到积木转换
IR	Intermediate Representation	中间表示（插件开发用）
TTS	Text-to-Speech	语音合成
LSP	Language Server Protocol	语言服务器协议
GPIO	General-Purpose Input/Output	通用输入输出引脚

第一章　绪论

1.1　研究背景与意义

程序设计能力与计算思维已成为数字时代公民素养的重要组成部分[8]。Python 凭借语法简洁与生态丰富，已成为程序设计入门课程的主流语言，但初学者仍普遍面临三重障碍：文本语法与缩进的严格约束、本地开发环境的搭建成本，以及从具象操作到抽象代码的认知跳跃。以 Scratch 为代表的图形化编程通过拖拽积木显著降低了入门门槛[1]，建构主义学习理论亦为其提供了经典支撑[4]；然而大量研究表明，学习者从积木迁移到文本语言时存在明显的“断层”：积木系统通常不是为产出真实工程语言而设计，缺少与文本代码的双向对照[2,3]。与此同时，创客与物联网教育要求学习者将程序作用于真实世界，而 MicroPython[11]虽将 Python 带入微控制器，其开发仍依赖专业工具链，与图形化教学环境长期割裂。

与此同时，生成式人工智能正在重塑编程实践。以大语言模型为核心的编程助手已能根据自然语言描述直接生成代码，显著提升专业开发者的生产率[6]。但将这类能力直接引入教学情境存在结构性风险：助手输出的是最终代码文本，初学者既难以判断其正确性，也难以将生成结果与自身认知结构建立联系，“黑箱式”的答案交付反而可能挤占必要的试错与建构过程。

在此背景下，本文探索一条折中路线：让人工智能以图形化积木为中介参与编程学习全过程——它的每一个动作都表现为工作区中可见、可撤销的积木操作；更进一步，让智能体具备按需“制造工具”的能力，即根据学习者需求自主开发新的积木插件。基于这一思路，本文设计并实现了桌面教学平台 Visual Python，统一支撑 Python 编程、数学动画创作与三类主流开发板的硬件编程。[1]

1.2　研究现状与问题提出

[1]作者简介：谭伟，重庆市育才中学校，研究方向为图形化编程与人工智能教育，E-mail：1010693922@qq.com。通信作者：谭伟。

图形化编程工具方面，Scratch 构建了面向儿童创意表达的完整生态[1]；Google Blockly 以库的形态被广泛用于构建各领域积木语言[5]，并支持将积木导出为多种文本语言。针对“积木到文本”的过渡，Weintrop 等的实证研究比较了积木式与文本式环境在中学情境下的学习效果与学习者偏好[2]，Bau 等系统论述了“可学习的编程”在积木范式下的设计原则[3]。硬件教育方面，MicroPython 与 ESP32 系列微控制器已成为创客教育的主流组合[11]，但缺少面向初学者的图形化入口。生成式 AI 辅助编程方面，以 Codex CLI 为代表的智能体已能在终端环境中自主完成多步骤编程任务[6]；Manim 动画引擎为数学知识的多媒体表达提供了专业工具[7]。然而既有工具普遍缺少三方面能力：与真实文本语言的双向、实时对照；人工智能对图形化编辑与工具扩展的深度参与；以及将积木编程延伸到真实硬件的一体化环境。由此提出本文的三个研究问题：

RQ1：如何设计一个在本地桌面环境内贯通“积木编辑—代码生成—运行验证—动画产出—硬件控制”全流程的平台架构？

RQ2：人工智能应经由何种机制参与图形化编程与工具扩展，才能兼顾支持效率与学习者的知情权、控制权？

RQ3：平台如何支撑标准化的教学流程，并为学科内容与硬件扩展提供可持续的生态机制？

1.3 研究内容与创新点

本文的研究内容包括三个方面：（1）平台总体架构与核心子系统设计，涵盖四层架构、四类工作台、积木引擎、语言服务、桌面命令与硬件服务；（2）人工智能深度参与的协同机制设计，包括代码优先、经确定性转换与隔离校验间接编辑工作区，以及自主插件开发闭环；（3）面向教学的应用模式设计与插件生态构建。对应的创新点有四项：

（1）代码优先、积木落地的透明人机协同机制。人工智能先生成完整的 Python/MicroPython 程序，再由确定性转换器编译为真实积木树，经隔离工作区校验后一次提交；代码仅作为可编译的中间表示，学习者看到、编辑和撤销的始终是积木。转换器无法覆盖的语句会显式报错并定位行号，智能体可修正代码或回退到结构化积木树，从机制上兼顾生成覆盖率与学习者的知情权、控制权。

（2）人工智能自主插件开发闭环。智能体依据自然语言需求，经“规划—中间表示编译—静态验证—宿主运行检查—用户确认安装—失败回滚”流程产出可安装的积木插件，且插件目标覆盖 Python 与三类开发板工作台——平台能力可由人工智能在运行期按需扩展，而非固定于发行版本。

（3）图形化硬件编程工作流。将 ESP32-S3、ESP32-WROOM、ESP8266 三类 MicroPython 开发板纳入同一积木环境：板卡引脚档案与积木级引脚校验、固件烧录、串口会话、设备文件管理与离线模拟器一体化，并在 ESP32-WROOM 与 ESP8266-ESP12E 上完成真机冒烟测试。

（4）本地全流程一体化中文教学环境。积木与 Python/MicroPython 代码双向实时对照、Pyright 诊断的中文归因、Manim 动画渲染与多角色配音、.vpy 工程管理在四类工作台间共享同一交互范式，全部核心功能可离线运行。

1.4 论文组织结构

本文共六章：第一章绪论；第二章综述相关研究；第三章阐述平台总体设计（架构如图 3-1 所示）；第四章报告平台实现与典型案例（工作台界面如图 4-1、AI 工作流如图 4-2 与图 4-3、教学模式如图 4-4 所示）；第五章讨论平台特色、工具对比与有效性威胁；第六章总结全文。附录 A 给出实现度量明细。本文撰写遵循 GB/T 7713.2—2022[9]，参考文献著录遵循 GB/T 7714—2015[10]。

第二章　相关工作

2.1　图形化编程与积木—代码过渡

Scratch 确立了“积木即程序”的范式，其设计目标是支持儿童的创意表达而非产出工程代码[1]。Papert 的建构主义理论指出，学习者在“可触摸”的知识建构过程中形成更深刻的理解[4]，积木界面正是这一思想在编程教育中的具象化。Blockly 作为可嵌入的积木框架，支持生成包括 Python 在内的多种文本语言[5]，为构建面向真实语言的积木环境提供了基础。在积木—文本过渡研究方面，Weintrop 等对中学生开展的对照研究发现，积木式环境在入门阶段具有显著优势，且学习者在掌握积木后向文本过渡的意愿与能力均得到改善[2]；Bau 等进一步提出“可学习的编程”设计原则，强调即时反馈、分步构建与渐进披露[3]。上述工作共同指向一个设计要求：面向 Python 教学的积木环境应提供与文本代码的紧密对照，而非孤立的积木沙盒。

2.2　生成式人工智能辅助编程与创客硬件

大语言模型驱动的编程智能体已从代码补全发展为可自主规划与执行多步任务的代理。以 Codex CLI 为代表的终端智能体能够理解任务描述、调用工具、读写文件并运行验证[6]，其“应用服务器”形态支持进程内嵌入，为第三方平台集成提供了条件。然而此类智能体的输出对象是代码文本与文件系统，天然面向专业开发者；教学场景需要“代理性能”与“教学透明性”的平衡：智能体的修改应可被学习者审视、可与学习者手工作业共存、可在不满意时整体回退。硬件教育方面，MicroPython 将 Python 3 引入微控制器[11]，与 ESP32 系列结合已成为创客教育主流，但其开发依赖 REPL 终端与专业 IDE，初学者门槛较高；将硬件控制封装为积木并打通固件烧录与串口运行，是弥合这一鸿沟的可能路径。Manim 动画引擎[7]在数学可视化领域应用广泛，但命令行式工作流对初学者并不友好，将其封装为积木并打通渲染与预览，同样有助于降低创作门槛。

2.3　研究现状述评

综上，已有研究的贡献在于确立了积木范式的教学价值、智能体的工程能力与 MicroPython 的硬件可行性；不足则集中在三点：其一，积木环境与真实文本语言的对照深度不足，缺少语言服务级诊断与到硬件的延伸；其二，智能体与图形化编程环境的集成尚属空白，AI 辅助大多止步于对话问答，更不具备为平台扩展工具的能力；其三，缺少面向中文教学情境、覆盖“数据处理—动画创作—配音表达—硬件控制”的一体化本地环境。本文工作即针对上述缺口展开。

第三章　平台设计

3.1　设计目标与原则

平台的设计遵循四项原则：（1）学习者主体原则，一切智能辅助均以不剥夺学习者控制权为前提；（2）图形化与代码协同原则，积木与 Python/MicroPython 代码双向实时对照，促进有意迁移；（3）人机协同原则，人工智能以可见、可审批、可撤销的方式参与编辑，并可在受控流程下扩展平台能力；（4）开放扩展原则，学科内容与硬件支持均以插件与工作台形式组织，核心稳定而生态可生长。

3.2　总体架构

平台采用四层架构，如图 3-1 所示。用户界面层基于 React 19 构建中文工作台，提供积木工作区（Blockly 13）、代码视图（Monaco）、三面板控制台（输出 / 问题 / 预览）、AI 助手面板、配音工作台与硬件工作台（开发板选择、串口连接、设备文件管理）；四类工程工作台——Python（含 Manim）、ESP32-S3、ESP32-WROOM、ESP8266-ESP12E——共享同一交互范式。前端核心层实现积木引擎与 Python/MicroPython 代码生成器、语言服务客户端、智能体框架（agentV2，含代码转换与插件创作）、硬件积木与板卡引脚档案（119 种积木，逐引脚标注可用功能与保留状态），以及插件系统。桌面服务层以 Tauri 2（Rust）实现，向前端暴露 79 项命令，涵盖 Python 运行、Manim 渲染、语音合成与克隆、进程内嵌入的 Codex 智能体、工程与插件管理，以及硬件服务（串口会话、MicroPython 固件烧录、设备文件读写与离线模拟器）。本地运行时与外部服务层包括本地 CPython 运行时（manim、numpy、pandas、matplotlib）、内置打包的 Pyright 语言服务、云端语音合成服务，以及经串口连接的开发板硬件。分层之间以事件与命令调用解耦，除语音合成外全部核心功能可在无网络环境完成。

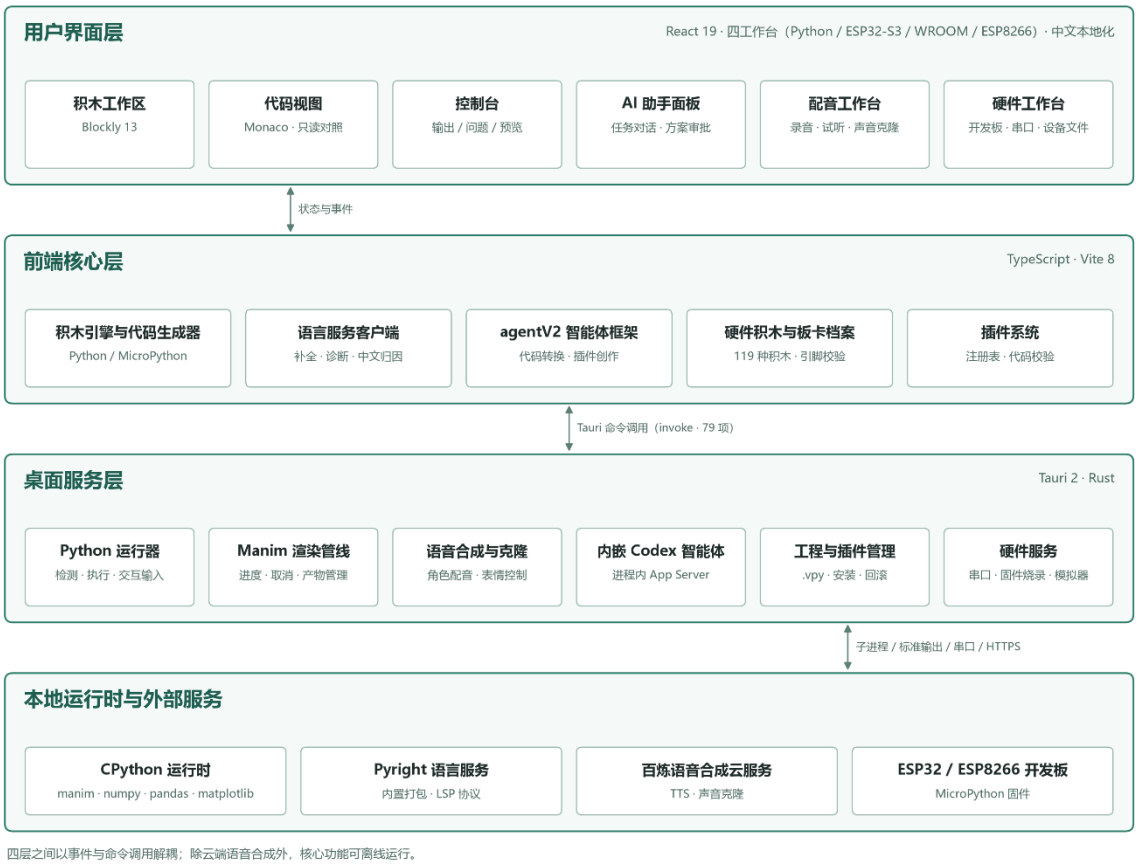


图 3-1　Visual Python 平台总体架构

3.3　积木体系与代码生成

平台共实现 601 种积木，按“核心语言 + 学科插件 + 硬件”三级组织，如表 3-1 所示。核心语言积木覆盖变量、运算、控制流、函数、输入输出与语法上下文校验；学科插件积木封装领域库常用调用；硬件积木覆盖 GPIO、PWM、模拟输入、I2C/SPI/UART 总线、WiFi、蓝牙、MQTT、ESPNOW、OLED/LCD1602 显示、DHT 温湿度、超声测距、舵机、步进电机、NeoPixel 彩灯、RFID、RTC 等外设。代码生成器随积木编辑实时产出规范 Python 或 MicroPython 源码，并保持积木与代码行双向映射；硬件积木受板卡引脚档案约束，非法引脚在编辑期即被标注。语言服务客户端将 Pyright 诊断归因为中文提示；对已有代码，平台提供反向解析器将其转换回积木，形成“代码→积木”（code-to-block，C2B）的逆向通道。

表 3-1　积木体系构成

模块	积木数量	主要内容
核心语言积木	203	变量、运算、控制流、函数、输入输出
Manim 插件	144	场景、图形、动画、坐标轴、配音
硬件积木	119	GPIO、总线、网络、传感与执行器
NumPy 插件	71	数组创建、运算、统计
Pandas 插件	35	数据结构、筛选、汇总
Matplotlib 插件	29	各类图表绘制与样式
合计	601	—

如表 3-1 所示，硬件积木（119 种）构成了覆盖创客教育常见外设的第三极，使同一积木范式从屏幕延伸到物理世界。

3.4　积木编辑协同机制设计

针对 RQ2 的编辑维度，平台最初采用“积木编辑计划”（blockPlan）作为中间表示，但在实测中发现：模型需在数百种积木中精确选择类型、字段与连接，目录截断、动态字段与 mutation 契约缺失会使长计划整体失败，且一次失败即回滚全部操作，生成稳定性不足。为此，平台将协同机制迭代为“代码优先、积木落地”：智能体提交完整的 Python 或 MicroPython 程序；宿主按工作台调用 manimPythonToWorkspace 或 micropythonToWorkspace，以确定性规则把代码解析为积木树；随后在隔离工作区加载该树，校验积木上下文约束，并由积木反向生成代码再次校验；全部通过后一次性提交，保留工作区原生撤销栈。转换器对无法无损映射的语句显式报错并给出源码行号，智能体据此修正代码，或改用 apply_block_program 提交嵌套积木树。该机制把代码降格为中间表示，最终进入学习者视野的仍是积木；同时，AI 的生成能力不再受积木目录长度与字段契约完整性的直接限制。

3.5　AI 自主插件开发机制设计

针对 RQ2 的工具扩展维度，平台赋予智能体“制造工具”的能力：依据学习者以自然语言表达的新积木需求，自主完成插件开发。如图 4-3 所示，流程为七步闭

环：①智能体先读取插件契约与官方开发指南，并确认目标工作台（Python 或三类开发板，避免将硬件插件误装入 Python 工程）；②规划插件蓝图；③将插件中间表示（IR）编译为 plugin.json、codegen.json 与入口源码；④静态验证积木定义、工具箱引用与生成代码语法；⑤宿主运行检查——在受控沙箱中实际加载执行插件，仅当检查就绪方可进入安装；⑥用户确认，这是安装前的唯一授权点，拒绝即终止；⑦安装后新积木即进入工具箱供所有工程使用。验证或运行失败时，智能体依据诊断信息以最小改动修订草稿后重试；安装失败时原插件文件自动备份、支持整体回滚。该机制使平台能力不再固定于发行版本，而可随教学需求在运行期生长。

3.6 评估指标设计

为支撑后续教学实验，平台在设计阶段即约定评估指标。设某任务的标准参考解答所含积木操作集合为 B_{ref}，学习者在平台中完成的积木操作集合为 B_{ai}（含本人手工与审批通过的 AI 生成操作），定义积木覆盖率如式(3-1)所示：

$$Cov = \frac{|BB_{ai} \cap B_{ref}|}{|B_{ref}|} \times 100\% \tag{3-1}$$

式(3-1)用于刻画任务完成度；结合渲染成功率、代码诊断清零率、对话轮次与硬件运行通过率等过程指标，构成多维评估框架。平台构建了覆盖四类工作台的 32 项 AI 积木生成基准任务（每类 8 项，从单一对象创建到多对象多动画组合递进），并在第 5.3 节报告首轮诊断性实测。本文报告平台的设计与实现，指标的教学实证留待后续研究。

第四章　平台实现与典型案例

4.1　实现规模与质量保障

平台前端基于 TypeScript 与 Vite 8 构建，非测试源码约 5.0 万行；Rust 桌面端（含硬件服务）约 1.5 万行；配套 127 个 Vitest 测试文件、1285 项用例，覆盖积木生成、代码到积木转换、插件注册与校验、硬件积木引脚规则、固件目录与配音预处理等核心模块。完整的度量明细见附录 A（表 A-1）。

4.2　工作台交互实现

Python 工作台采用可折叠的三栏布局：左侧为积木工作区，工具箱按“开始 / 文本 / 逻辑 / Manim / 变量 / NumPy / Matplotlib / Pandas”分类并可检索；右侧为实时生成的 Python 代码，顶部场景页签支持多文档管理；底部控制台在输出、问题与预览之间切换。学习者拖动积木，右侧代码即时更新；在分屏模式下，积木结构与对应代码行保持同步高亮，形成逐块对照的学习体验，界面如图 4-1 所示。

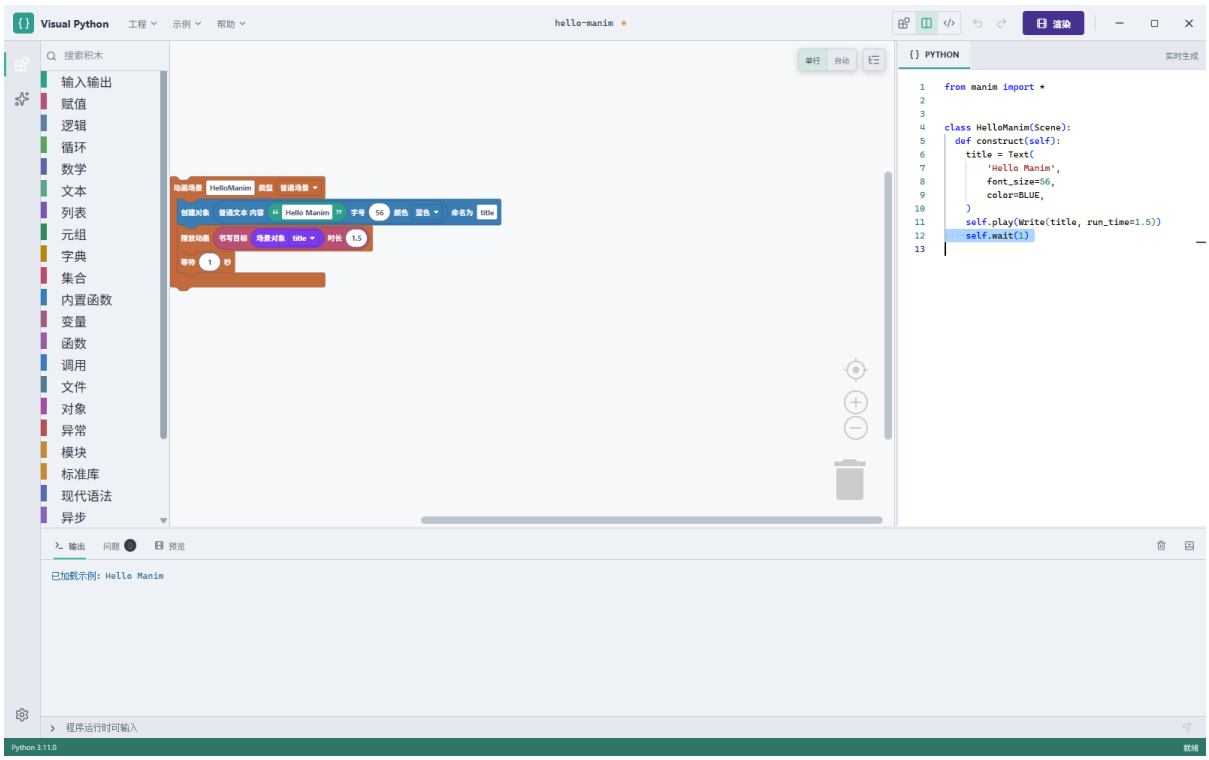


图 4-1　Visual Python 工作台分屏视图：积木与 Python 代码实时对照

4.3 AI 辅助积木编写实现

图 4-2 给出 AI 辅助积木编写的完整链路：学习者在 AI 助手面板以自然语言描述任务，智能体经任务理解后调用 apply_python_as_blocks，提交一段完整的 Python（Manim）或 MicroPython（硬件）程序；宿主按当前工作台选择 manimPythonToWorkspace 或 micropythonToWorkspace，把程序解析为真实积木树；随后在隔离工作区中加载该树，校验积木上下文与连接约束，并由积木反向生成代码、执行静态校验。只有全部检查通过，候选工作区才一次性提交到当前工作区，因此学习者看到的每一步都是积木，且可整体撤销。若转换器遇到无法无损映射的语句，会返回带源码行号的错误；智能体可修正代码后重试，或改用 apply_block_program 提交嵌套积木树。转换结果中的 fallbackCount 用于提示仍以原始语句积木承载的片段，避免“看似转换成功、实际丢失结构”。

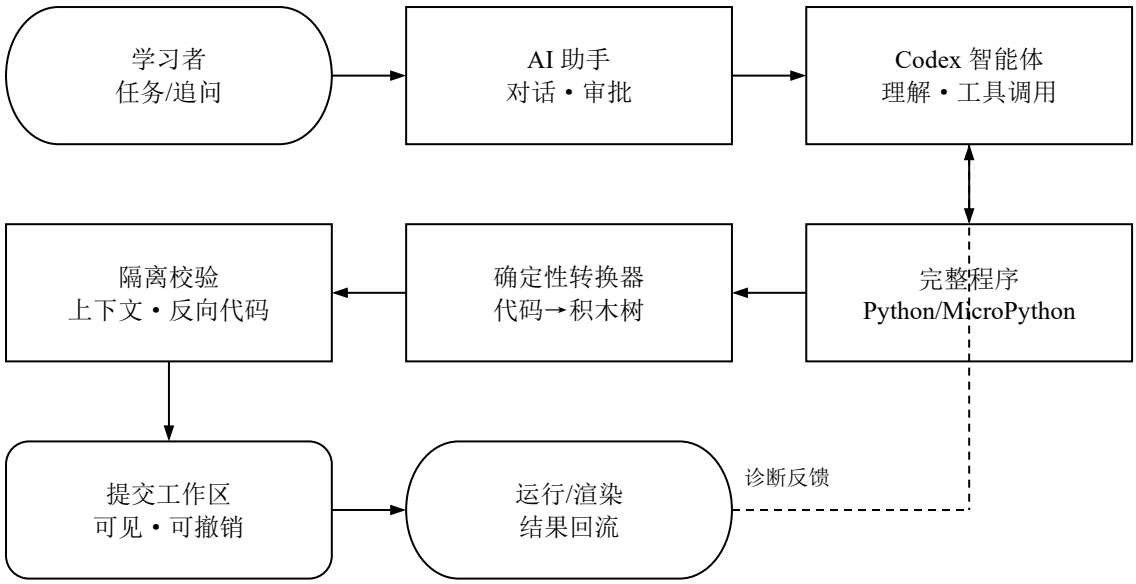


图 4-2 AI 辅助积木编写工作流

4.4 AI 插件开发实现

第 3.5 节设计的插件开发闭环已完整实现为智能体工具链（如图 4-3 所示）：规划与编译阶段对应 create_plugin_plan、create_plugin_draft 等工具，IR 草稿经 apply_plugin_plan 支持积木级的增删改；验证阶段由 validate_plugin_draft 与宿主运

行检查工具完成；安装由 install_plugin_draft 在用户确认后写入本地插件目录。智能体被约束“以工具返回的契约为准、不凭记忆编造宿主接口”，草稿与计划仅以工具签发的标识为准。实测中，学习者以“帮我做一个××积木包”类需求驱动智能体即可获得经双重验证的新积木；插件目标可为 Python 或三类开发板工作台。

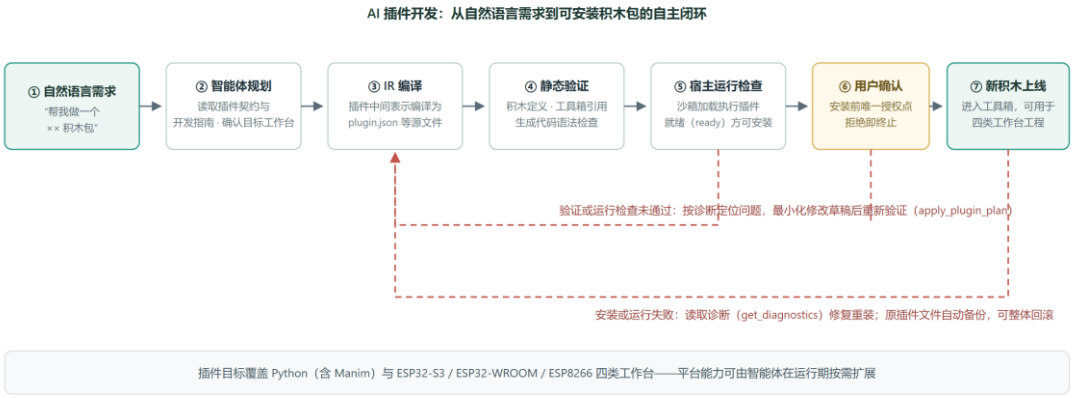


图 4-3　AI 插件开发流程

4.5　动画渲染与配音工作流

Manim[7] 工作流将“渲染”收敛为工具栏单一按钮：平台自动提取场景名、调用本地渲染管线、解析进度并支持取消，完成后在预览面板内嵌播放器展示视频。入门案例的一次完整实测中，示例场景渲染耗时 75 秒，输出保存于工程媒体目录，预览面板呈现“Hello Manim!”标题写入动画并可交互回放。配音工作流支持多角色中文配音与语音克隆，配音积木直接嵌入动画时间线，实现“动画＋解说”的一体化创作。

4.6　硬件编程工作流

硬件创作与 Python 编程共享同一积木范式。新建工程时选择开发板（ESP32-S3、ESP32-WROOM 或 ESP8266-ESP12E）即进入硬件工作台：工具箱呈现硬件分类积木，顶部提供串口连接、运行与停止；积木生成的 MicroPython 代码可一键经串口在开发板上运行，设备文件面板支持浏览、上传、读取与删除板载文件；未持有实物的学习者可切换到内置模拟器离线运行验证逻辑。固件烧录对话框提供各板卡配套的 MicroPython 固件目录与芯片系列校验，一站式完成“空板→可编程”准备。

积木层的引脚校验依据逐板引脚档案（标注每个引脚的数字/模拟/PWM/触摸/保留属性）在编辑期拦截非法引脚，从源头减少接线错误。真机冒烟测试在 ESP32-WROOM 与 ESP8266-ESP12E 上验证了板载 LED、PWM、模拟输入、I2C LCD1602 与 BH1750 光照传感器的端到端流程（ESP32-S3 已支持但尚无真机记录，见第 5.3 节）。

4.7 教学应用模式

面向课堂教学，平台支持六环节人机协同教学模式，如图 4-4 所示：教师负责情境创设、方案把关与多元评价；学习者承担方案设计、积木搭建（或审批智能体方案）、代码对照理解、运行验证与反思迁移；平台与智能体提供示例工程、积木文档、转换校验、实时诊断、渲染与硬件运行反馈。其中“积木搭建”环节允许人工搭建与 AI 生成交替进行；“运行验证”环节的错误信息回流智能体可自动定位修正，形成迭代闭环。



图 4-4　人机协同图形化编程教学模式

4.8 插件体系与生态

平台以插件为学科扩展的唯一通道，且扩展途径有二：人工智能自主开发（第 3.5、4.4 节）与人工开发。人工路径下，外部插件以清单文件描述积木、工具箱、Python 依赖与 AI 能力声明，支持 ZIP 包或目录安装；平台对插件生成的代码与工具箱引用进行静态校验，安装失败可自动回滚。随平台发布的 Turtle 海龟绘图示例插件（40 种积木）验证了第三方开发路径的可行性；配套的插件开发指南与积木文档工具链（自动产出 144 个 Manim 积木的截图与目录）构成了插件生态的基础设施。

第五章　讨论

5.1　特色分析

针对三个研究问题，平台的特色可概括为四点。其一，本地全闭环：从积木编辑、代码生成、静态诊断到运行、动画渲染与硬件控制，均在桌面环境内完成，避免在线工具的网络依赖与课堂环境的不可控性。其二，AI 透明协同：编辑层面，AI 经“代码优先、积木落地”的转换路径作用于积木层，代码只是中间表示，操作可见、可审批、可撤销；扩展层面，AI 开发插件须经静态验证、宿主运行检查与用户确认三道闸门，失败可回滚——学习者的程序结构认知始终由图形化媒介承载，平台能力生长亦处于学习者监督之下。其三，双向对照与诊断闭环：积木与 Python/MicroPython 代码的实时映射叠加 Pyright 诊断的中文归因与硬件引脚校验，使“看得见的结构”“读得懂的报错”“接不错线”同时可得，支撑从积木到文本的有意迁移[2,3]。其四，屏幕到硬件的连续性：同一积木范式覆盖动画创作与三类 MicroPython 开发板，配合离线模拟器，创客教学可在无硬件条件下先行展开。

5.2　与典型工具的能力对比

表 5-1 从八个维度比较本平台与三类典型工具。与 Scratch 相比[1]，本平台以 Python 为生成目标并提供双向对照与硬件通道；与 Blockly 在线示例相比[5]，本平台提供桌面级运行、渲染、硬件与智能体集成；与代码助手类工具相比[6]，本平台面向初学者、以积木为 AI 作用面且 AI 可扩展平台自身。

表 5-1　本平台与典型编程工具能力对比

能力维度	Scratch	Blockly 在线示例	代码助手类工具	本平台
图形化积木编辑	有	有	无	有（四类工作台）
实时生成 Python	无	部分	直接生成文本	有，双向映射
AI 参与编辑	无	无	直接改写代码	代码优先转换，可审批
AI 自主开发插件	无	无	无	有（双重验证＋回滚）
本地运行与动画渲染	无	部分	依赖终端	有（含 Manim 渲染）

能力维度	Scratch	Blockly 在线示例	代码助手类工具	本平台
硬件编程	需扩展硬件	无	无	三类 ESP 开发板＋模拟器
中文教学情境	部分	无	部分	全面（界面、诊断、文档）
插件扩展生态	有	有	无	有（人工＋AI 双通道）

如表 5-1 所示，本平台的差异化定位在于“图形化、AI 透明协同与自主扩展、本地闭环（含硬件）”的交集。

5.3 有效性威胁与局限

本文属于平台设计与实现研究，存在以下局限。内部效度方面，第 4.5 节的渲染耗时为单机单次实测（75 秒），受硬件与场景复杂度影响，仅作可用性佐证而非性能结论；AI 积木生成与插件开发质量依赖外部大模型，32 项基准任务的首轮诊断性实测在 qwen3.8-flash 上通过 12 项（37.5%），其中 Python 工作台 5/8、ESP32-S3 5/8、ESP32-WROOM 2/8、ESP8266 0/8；失败主要集中在转换器未覆盖的库调用、长程序超时与原始语句积木回退。该结果仅用于定位工程缺口，不构成对平台教学效果的推断。外部效度方面，平台深度适配 Windows 桌面环境；真机冒烟测试覆盖 ESP32-WROOM 与 ESP8266-ESP12E，ESP32-S3 的支持基于引脚档案与固件目录但尚无真机运行记录；界面与文档面向中文教学情境，跨语言适应性有待检验。构念效度方面，式(3-1)定义的覆盖率指标尚未经教学实证标定。结论效度方面，本文未开展课堂对照实验，教学效果主张均以设计论证与功能实测为限。上述局限构成后续实证研究的动机。

第六章　结论与展望

6.1　研究结论

针对 RQ1，平台以四层架构与 79 项桌面命令实现了“积木—代码—运行—渲染—硬件”的本地全流程闭环，四类工作台共享同一交互范式；针对 RQ2，提出了 AI 经“代码优先、积木落地”转换间接编辑与经七步闭环自主开发插件的双重协同机制，兼顾了支持效率、能力扩展性与学习者控制权；针对 RQ3，平台以插件体系承载学科与硬件扩展（601 种积木，含 119 种硬件积木），并以六环节教学模式、32 项基准任务与配套文档工具链支撑课堂应用与质量评估。

6.2　主要贡献

本文贡献包括：（1）提出并实现了“AI 生成代码、代码编译为积木”的教学型人机协同机制，用确定性转换与隔离校验替代让模型直接猜测积木 schema，使代码成为可编译的中间表示而非最终交付物；（2）提出并实现了人工智能自主插件开发闭环，使图形化编程平台的能力可由智能体在运行期受控扩展；（3）将三类主流 MicroPython 开发板的硬件编程纳入积木环境，形成“引脚校验—固件烧录—串口运行—离线模拟”的完整创客教学工作流，并在 ESP32-WROOM 与 ESP8266-ESP12E 上完成真机冒烟验证；（4）给出了从架构到插件规范的完整实现与可复现的度量基准，为同类教学平台建设提供参考。

6.3　研究局限与未来工作

后续工作将沿三个方向推进：其一，依托 32 项基准任务与式(3-1)指标开展课堂对照实验，实证检验平台对学习成效、迁移能力与硬件创作兴趣的影响；其二，扩展 AI 协同形态，包括面向教师的学情分析与学习轨迹回放；其三，推进跨平台适配、ESP32-S3 真机验证与插件生态的社区化运营。

致　谢

感谢参与平台试用与反馈的师生，以及重庆市育才中学校对本研究的支持。

声　明

伦理声明：本文为平台设计与实现研究，未涉及人类被试；后续教学实证将另行接受伦理审查并获取知情同意。

利益冲突：全体作者声明无利益冲突。

数据与材料可用性：平台源代码与本文全部配图素材见项目仓库（地址见随论文提交的补充材料）；附录 A 度量数据可由仓库脚本复现。

人工智能使用披露：本文写作过程中使用大模型辅助文献梳理与文字润色，全部技术内容、实测数据、图表与结论由作者核实并负责。

参考文献

[1] Resnick M, Silverman B, Pape Y B, et al. Scratch: programming for all[J]. Communications of the ACM, 2009, 52(11): 60-67.

[2] Weintrop D, Wilensky U. To block or not to block, that is the question: students' perceptions of blocks-based programming[C]//Proceedings of the 14th International Conference on Interaction Design and Children. New York: ACM, 2015: 199-208.

[3] Bau D, Gray J, Kelleher C, et al. Learnable programming: blocks and beyond[J]. Communications of the ACM, 2017, 60(6): 74-80.

[4] Papert S. Mindstorms: children, computers, and powerful ideas[M]. 2nd ed. New York: Basic Books, 1993.

[5] Fraser N. Blockly: a visual programming editor[CP/OL]. Google. [2026-10-24]. https://developers.google.com/blockly.

[6] OpenAI. Codex CLI[CP/OL]. GitHub. [2026-10-24]. https://github.com/openai/codex.

[7] The Manim Community. Manim: an animation engine for explanatory math videos[CP/OL]. [2026-10-24]. https://www.manim.community.

[8] Wing J M. Computational thinking[J]. Communications of the ACM, 2006, 49(3): 33-35.

[9] 全国信息与文献标准化技术委员会. 学术论文编写规则: GB/T 7713.2—2022[S]. 北京: 中国标准出版社, 2022.

[10] 全国信息与文献标准化技术委员会. 信息与文献 参考文献著录规则: GB/T 7714—2015[S]. 北京: 中国标准出版社, 2015.

[11] MicroPython Developers. MicroPython: a lean implementation of Python 3 designed to run on microcontrollers[CP/OL]. [2026-10-24]. https://micropython.org.

附录 A　平台实现度量明细

平台实现度量由仓库脚本统计（统计日期：2026-10-24，代码基线含硬件分支），如表 A-1 所示。

表 A-1　平台实现度量明细

度量项	数值	说明
前端非测试源码	50 226 行	TypeScript/TSX，不含测试
Rust 桌面端源码	14 823 行	含硬件服务模块（串口/固件/模拟）
桌面命令	79 项	Python、渲染、TTS、插件及 11 项硬件命令
积木总数	601 种	见表 3-1 分项（硬件 119 种）
工程工作台	4 类	Python（含 Manim）与三类 ESP 开发板
测试文件	127 个	Vitest，源码同目录放置；1285 项用例
AI 积木生成基准任务	32 项	四类工作台各 8 项
真机冒烟测试	2 种板卡	ESP32-WROOM、ESP8266-ESP12E
Manim 积木文档截图	144 张	文档工具链自动生成
外部示例插件	1 个	Turtle，40 种积木

如表 A-1 所示，平台以中等工作量实现了较高的功能密度，测试覆盖、基准任务与文档自动化程度较高。